PheW TuBe: High-Performance Video Streaming Platform

Comprehensive Viva Voce Defense Guide & Live Demonstration Scripts

Group G-15 • Project ID: G15-PID-17 • Course: FEE-II (25CSE0203) • Chitkara University

1. The 1-Minute Thesis Hook & The 5 Crucial Evaluator Questions

Examiner Question: *'Tell me about your project in 1 minute. Justify your title, what gap you filled, and what happens after applying it.'*

Topic 1: Viva Title Justification
PheW TuBe stands for an ad-free, high-performance in-site video streaming Single Page Application. It eliminates disruptive external site redirects, enabling continuous 1080p playback, hands-free Web Speech API search, and granular YouTube settings within an authentic React 18 ecosystem.

Topic 2: 5-Sentence Pitch & The Technical Gap Overcome
1. *What this is:* A responsive, component-driven video streaming SPA replicating modern YouTube architecture.
2. *Why we chose this:* Traditional clones either freeze when scaling feeds or lazily kick users off-site to YouTube via external window redirects.
3. *The Gap Filled:* Built a true in-app streaming player on /watch/:id powered by dynamic useParams() with zero full-page reloads.
4. *How in One Line:* Decoupled video state via Context API, achieved DOM focus with useRef, and integrated Web Speech API for hands-free voice search.
5. *Stack & Platform:* Engineered for laptop and mobile browsers using React 18, Vite 5, React Router v6, Tailwind CSS 3, and LocalStorage.

Topic 3: Target Customer Base & Real-World Use Case
• **Students & Programmers:** Distraction-free educational viewing (React, DSA, Latent episodes) with interactive timestamps.
• **Accessibility Users:** Hands-free query formulation via voice speech recognition without touching keyboards.
• **Content Viewers:** Fast, lightweight video consumption with instant dark/light theme switching and persistent watch history.

Topic 4: After Applying What Will Happen? (The Impact)
1. **Zero Context Loss:** 100% in-app media playback ensures user retention without navigating away from the platform.
2. **Instant State Hydration:** Likes, Watch Later bookmarks, and comments persist permanently across browser reloads via LocalStorage.
3. **Sub-1.2s Bundle Loading:** Optimized Vite 5 production build renders in under 1.5 seconds with zero layout shifts.

Topic 5: 1st Person's Job (Impression & Hook)
Do not read slides! Open the live project immediately on localhost:5173, look at the evaluators with confidence, and speak directly with technical authority.

2. 4-Teammate Live Demonstration Script (Step-by-Step)

Instead of boring PPT reading, transition seamlessly between all 4 teammates in under 3 minutes:

Teammate 1: Kishan (Project Lead & The Hook) — Voice Search & App Architecture

What to Do on Screen: Project running on localhost:5173. Click the glowing red mic icon on the navbar.

Exact Spoken Words:
"Good morning, respected evaluators. I am Kishan, presenting PheW TuBe (Group G-15). While ordinary clones freeze or redirect users to external YouTube tabs, we built an authentic, stateful Single Page Application in React 18 with zero external redirects.

First, notice our accessibility layer: when I click this mic icon on the navbar, our Web Speech API listener activates with an animated glowing red pulse ring. It converts my voice into text in real time, auto-fills the search input, and instantly filters our video catalog without touching the keyboard. Now, my teammate will demonstrate our protected authentication and LocalStorage state engine."

Teammate 2: Kanav Chadha — Login, Signup, Duplicate Guard & LocalStorage

What to Do on Screen: Click 'Sign In' button on top-right. Open Chrome DevTools (F12) -> Application -> Local Storage. Show 'phew_registered_users'. Attempt duplicate signup, then click 'Clear All Saved Profiles'.

Exact Spoken Words:
"Hello sir. I am Kanav. Our platform implements an authenticated session gatekeeper. Notice our AuthModal: if an unauthenticated guest tries to access protected features like 'Liked Videos' or 'Watch Later', our client-side Protected Route intercepts them.

In our Sign Up tab, if someone enters an existing account like 'Kishan', our validation engine catches the duplicate and triggers a toast notification: 'User already registered! Please sign in with your credentials.' Behind the scenes in DevTools Application tab, our accounts are hydrated directly in browser LocalStorage under 'phew_registered_users'. For quick testing and evaluation, we built this dedicated 'Clear All Saved Profiles' button which resets local storage instantly. Next, my teammate will show our in-site playback engine."

Teammate 3: Arsh Behl — In-Site Video Playback Engine (/watch/:id) & useRef Comments

What to Do on Screen: Click on any video card (e.g. Chai aur Code or GTA 5). Show URL changing to /watch/:id without page reload. Click Like button (increments). Type a comment and post it.

Exact Spoken Words:
"Respected jury, I am Arsh. In Phase 1, clicking a video redirected externally. In Phase 2, clicking any video transitions to our deep route /watch/:id using React Router v6's useParams hook with zero full-page reload.

Notice that the video plays live inside our 16:9 embedded player frame. All interactive controls are fully functional: clicking Like increments our counter and updates our global AppContext bus. Furthermore, in our comment thread, we use React's useRef hook to directly focus the input field upon interaction without triggering synthetic DOM re-renders. Users can post comments with instant avatar attribution. My teammate will now demonstrate our responsive architecture."

Teammate 4: Soumajit Majee — Multi-Device Responsiveness & Phase 2 Advancements

What to Do on Screen: Open DevTools (Ctrl+Shift+M). Select 'iPad Pro 13 (1032x1376)'. Then resize down to Mobile (375px). Show the sidebar collapsing into a mini-rail, and the 4-column feed transforming to 2-column then 1-column.

Exact Spoken Words:
"Good morning evaluators, I am Soumajit. Our platform is engineered with a strict Tailwind CSS 3 breakpoint matrix (sm, md, lg, xl, 2xl) ensuring zero horizontal scrollbars across all screen geometries.

On this iPad Pro 1032x1376 viewport, our expanded 240px sidebar automatically collapses into an ergonomic icon mini-rail, preserving maximum video viewing area while the card grid dynamically switches from 4 columns to 2 columns. On mobile viewports, it seamlessly adapts into a single-column touch stream.

In Phase 2, we completely transformed our app from static mock cards into a stateful SPA: adding dynamic React routing, protected auth guards, embedded video streaming, LocalStorage persistence, voice recognition, and infinite scroll pagination. Thank you!"

3. Classroom Viva Questions & Direct Model Answers (From Real Evaluations)

Direct, concise answers to all questions asked by the teacher in recent sessions:

Q1: 'Ek link di hai, usse open nahi karna, usse download karna hai HTML mein kaise karenge?'

Answer: HTML mein <a> anchor tag ke andar **download** attribute ka use karenge:
Download File.
Browser is attribute ko dekhte hi link ko navigate karne ke bajaye file ko directly user ke local storage/downloads folder me download kar deta hai.

Q2: 'Real DOM aur Virtual DOM mein kya difference hai? What is its purpose?'

Answer: • **Real DOM:** Browser ka actual HTML tree representation hota hai. Isme direct update karna bahut slow hota hai kyunki browser ko poora page re-layout aur re-paint karna padta hai.
• **Virtual DOM:** Real DOM ki lightweight in-memory JavaScript representation hoti hai (React memory me banata hai).
• **Purpose:** Jab state change hoti hai, React naya Virtual DOM banata hai, **Diffing Algorithm (Reconciliation)** se purane aur naye Virtual DOM ko compare karta hai, aur sirf unhi actual nodes ko Real DOM me patch karta hai jo change hue hain. Isse performance 10x improve ho jaati hai.

Q3: 'What are React Hooks? Kaun kaun se hooks use kiye hain?'

Answer: Hooks React 16.8 me introduce hue special functions hain jo functional components ko state aur lifecycle features use karne dete hain without writing class components.
Hamare project me used hooks:
• **useState:** Local component state store karne ke liye (search input, active tabs, modal open/close).
• **useEffect:** Side-effects handle karne ke liye (LocalStorage sync, API fetch, event listeners).
• **useRef:** Real DOM element ka direct mutable reference rakhne ke liye (comment input focus karna without re-render).
• **useContext (useApp):** Centralized global state access karne ke liye without prop-drilling.
• **useParams:** URL dynamic route params extract karne ke liye (/watch/:id).
• **useNavigate:** Client-side programmatic navigation perform karne ke liye.

Q4: 'Konsa hyper package install karna padta hai? What is the command to install packages?'

Answer: Node.js ke default package manager **npm** (Node Package Manager) ka use karte hain.
• **Command:** npm install <package-name> ya short form npm i <package-name>.
• Hamare project ke packages: react-router-dom (routing ke liye), react-icons (YouTube icons ke liye), react-toastify (toast popups ke liye), react-infinite-scroll-component (lazy loading pagination ke liye).

Q5: 'package.json kya hota hai aur package-lock.json se kya difference hai?'

Answer: • **package.json:** Project ki manifest file hoti hai jo project name, version, runnable scripts (dev, build), aur dependencies list karti hai.
• **package-lock.json:** Har ek installed package aur uske child dependencies ka exact tree structure aur cryptographic hash (integrity) record karti hai, taaki kisi aur system par npm install chalane par exact same versions install hon.

Q6: 'Boilerplate ki short form kya hai? React component boilerplate likhne ka shortcut kya hai?'

Answer: VS Code me ES7+ React/Redux extension ke dwara shortcut snippet code use hota hai:
• **rafce**: *React Arrow Function Component with Export* (Arrow function component banata hai aur end me default export karta hai).
• **rfce**: *React Functional Component with Export* (Regular function component banata hai).
• Project boilerplate scaffold karne ka command: npm create vite@latest.

Q7: 'Agar ek CSS file Navbar.js mein lagani hai, how will we do it? Kis naam ki file banayenge?'

Answer: 1. File ka naam **Navbar.css** (ya scoped CSS ke liye Navbar.module.css) banayenge.
2. Navbar.js ke sabse upar import karenge:
import './Navbar.css';
(Agar CSS Module ho toh: import styles from './Navbar.module.css'; aur JSX me className={styles.navbar} use karenge).

Q8: 'Components kya hota hai? Navbar agar banate hain toh React me use kaise karte hain?'

Answer: • **Component:** Reusable, self-contained UI building block hota hai jo HTML (JSX), CSS, aur JavaScript logic ko combine karta hai.
• **Usage:** Navbar.jsx me function banakar export default Navbar; karenge. Phir parent component (jaise App.jsx) me import karke custom tag ki tarah render karenge:
import Navbar from './components/Navbar';
JSX me: <Navbar />.

Q9: 'What is OOPs in FEE / JavaScript?'

Answer: JavaScript me Object-Oriented Programming (OOP) Prototype-based inheritance aur ES6 Classes ke dwara implement hoti hai:
1. **Class & Object:** class User { constructor(name) { this.name = name; } }
2. **Inheritance:** class Admin extends User { ... } (extends keyword).
3. **Encapsulation:** Private fields (#privateField) ya closures ke dwara data protection.
4. **Polymorphism:** Subclass dwara parent class ke method ko override karna.

Q10: 'slice() method vs splice() method in JavaScript?'

Answer: • **slice(start, end):** Array ka ek portion copy karke **naya array** return karta hai. Original array ko mutate NAHI karta (Immutable). Example: videos.slice(0, 8).
• **splice(start, deleteCount, items...):** Original array me se elements remove ya add karta hai. Yeh **original array ko mutate karta hai** (Destructive). React state me immutable hone ke kaaran slice() ya spread operator prefer kiya jaata hai.

Q11: 'React me DOM ka kya use kiya hua hai? JS ke konse features use kiye hain?'

Answer: • **React DOM:** Virtual DOM engine real DOM ke sath batching aur fiber reconciliation handle karta hai. Programmatic DOM access ke liye useRef hook use kiya hai comment box ko dynamically focus karne ke liye.
• **Modern JS Features:** Destructuring assignment (const { id } = useParams()), Arrow functions, Spread operator ([...prev, newItem]), Array methods (map, filter, sort, slice, find), Async/Await, Template literals, LocalStorage Web API, aur Web Speech API.